

GitHub dokumentacijos pagrindai – SafeStack

Paskirtis: praktinis, struktūrinis GitHub naudojimo ir saugumo aprašas SafeStack projektams.

Šis dokumentas aprašo procesus ir saugumo principus. Į jį negalima rašyti tikrų slaptažodžių, privačių raktų, API tokenų ar kitų paslapčių.

1. GitHub struktūra

- **Account (paskyra)** – žmogaus arba automatizacijos tapatybė.
- **Organization** – bendra projektų, komandų ir politikų erdvė, pavyzdžiui, kibernetinio-saugumo-sprendimai.
- **Repository (repo)** – vieno projekto kodas, dokumentacija, leidimai, Issues, Pull Requests ir Actions.
- **Branch** – atskira kodo versijos linija. main turi būti stabili.
- **Commit** – konkretus pakeitimas su autoriumi, data ir pasirinkamu kriptografiniu parašu.
- **Tag / Release** – pažymėta versija, kurią galima patikrinti ir atsekti.
- **Issue** – užduotis, klaida arba saugumo klausimas.
- **Pull Request (PR)** – kontroliuojamas pakeitimo kelias su peržiūra ir testais.
- **GitHub Actions** – CI/CD automatizacija, kuri vykdoma pagal YAML workflow failus kataloge `.github/workflows/`.

Rekomenduojama kiekvienam SafeStack projektui turėti `README.md`, `SECURITY.md`, `LICENSE`, testus, pakeitimų istoriją ir aiškų leidimo procesą.

2. Raktai ir kredencialai

2.1. SSH raktai Git operacijoms

SSH porą sudaro:

- **privatus raktas** – lieka tik savininko įrenginyje arba saugioje offline laikmenoje;
- **viešas raktas** (`.pub`) – įkeliamas į GitHub paskyrą arba naudojamas kaip deploy key.

Privatus raktas niekada nekeliamas į repo, Issue, chatą, Desktop bendrinamą katalogą ar CI logą. Rekomenduojama naudoti atskirą raktą kompiuteriui, automatizacijai ir leidimų pasirašymui. Jei raktas galėjo būti nukopijuotas, jį reikia pašalinti GitHub ir sugeneruoti naują.

Patikrinimas:

Ошибка! Не указано имя файла.



```
ssh -T git@github.com  
ssh-keygen -lf ~/.ssh/id_ed25519.pub -E sha256
```

GitHub serverio fingerprintą tikrinkite pagal oficialų sąrašą prieš pirmą yes atsakymą.

2.2. Commit ir release parašai

Commit ar leidimo parašas patvirtina, kad objektą pasirašė konkretus raktas, tačiau neįrodo, kad pats kodas yra saugus. Parašams galima naudoti SSH, GPG arba S/MIME. Viešas parašo raktas gali būti registre, o privatus – offline, su atsargine kopija ir aiškiu rotacijos planu.

SafeStack leidimams naudinga saugoti:

- projekto pavadinimą ir versiją;
- viešo rakto fingerprintą;
- pasirašytą hash manifestą;
- rakto būseną: active, revoked arba retired;
- rakto rotacijos datą ir priežastį.

2.3. Personal Access Token (PAT)

PAT yra paskyros vardu veikiantis kredencialas. GitHub rekomenduoja naudoti **fine-grained PAT**, kai jo pakanka, o klasikinius tokenus palikti tik išimtiniais atvejais.

Fine-grained tokenas turi būti apribotas:

1. viena organizacija arba savininku;
2. konkrečiais repo;
3. konkrečiais leidimais (Contents: read, Contents: write ir panašiai);
4. trumpiausiu praktišku galiojimo laiku;
5. organizacijos patvirtinimo politika, jei ji įjungta.

Klasikinis PAT gali suteikti daug platesnę prieigą prie visų paskyros repo, todėl yra rizikingesnis. Tokenai traktuojami kaip slaptažodžiai: jų neįrašome į kodą, URL, commitą, README ar terminalo istoriją.

Jei tokenas nutekėjo:

1. nedelsiant jį revoke GitHub nustatymuose;
2. patikrink audit log ir repo istoriją;
3. pakeisk susijusius API raktus ir paslaptis;
4. išvalyk paslaptį iš istorijos tik po revokacijos;
5. sukurk naują mažiausių teisių kredencialą tik jei jo tikrai reikia.

2.4. GitHub Apps

Ilgalaikiai integracijai ar organizacijos automatizacijai rinkis **GitHub App**, o ne asmeninį PAT. App leidžia pasirinkti konkrečius repo ir smulkiagrūdžius leidimus, naudoja trumpai galiojančius installation tokenus ir nėra pririštas prie vieno žmogaus darbo santykių.

2.5. Deploy keys

Deploy key yra SSH raktas, pririštas prie vieno repo. Skaitymo teisė turėtų būti numatyta pagal nutylėjimą; rašymo teisė suteikiama tik tada, kai būtina. Vieno privataus deploy rakto nenaudok keliems projektams.

3. API naudojimas

GitHub REST API naudoja bazinį adresą:

Ошибка! Не указано имя файла.

`https://api.github.com`

Reikalavimai patikimam klientui:

- naudoti oficialią API versiją ir aiškų Accept header;
- autentikuoti per GitHub App arba mažiausių teisių fine-grained tokeną;
- tokeną perduoti per aplinkos kintamąjį arba paslapčių saugyklą, ne per šaltinio kodą;
- tikrinti HTTP statusą ir JSON klaidas;
- saugiai apdoroti pagination;
- laikytis x-ratelimit-*, retry-after ir x-ratelimit-reset headerių;
- naudoti exponential backoff ir nebandyti agresyviai kartoti užklausų;
- registruoti įvykį be tokeno, Authorization headerio ar privačių duomenų.

Paprastas saugus šablonas:

Ошибка! Не указано имя файла.



```
: "${GITHUB_TOKEN:?GITHUB_TOKEN is required}"
curl --fail-with-body \
  -H "Accept: application/vnd.github+json" \
  -H "Authorization: Bearer ${GITHUB_TOKEN}" \
  -H "X-GitHub-API-Version: 2022-11-28" \
  "https://api.github.com/repos/OWNER/REPO"
```

API limitai priklauso nuo autentikacijos. Neautentifikuotoms REST užklausoms taikomas mažas limitas, autentifikuotiems naudotojams paprastai taikomas didesnis limitas, o GitHub App installation tokenas gali būti tinkamesnis automatizacijai. Limitas nėra leidimas apeiti prieigos kontrolę.

4. GitHub Actions ir paslaptys

Paslaptis laikyk:

- repository secret, jei ją naudoja vienas repo;
- environment secret, jei reikia approval prieš production;
- organization secret, jei keli repo turi tą pačią paslaptį;
- išorinėje paslapčių saugykloje, jei reikia audito, rotacijos ir centralizuoto valdymo.

Workflow turi naudoti mažiausias permissions:

Ошибка! Не указано имя файла.

```
permissions:
  contents: read
```

Rašymo teisę (contents: write) suteik tik konkrečiam workflow, kuriam jos reikia. Trečiųjų šalių Actions fiksuok patikima versija arba commit SHA; nepasitikėk akiai @main. Neįterpk paslapties į run: komandą, artefaktus ar debug logus.

GITHUB_TOKEN yra patogus repo ribų veiksmams. Veiksmams už workflow repo ribų reikia aiškiai suprojektuoto GitHub App arba kito riboto kredencialo.

5. Repo apsaugos nustatymai

Minimalus viešo projekto profilis:

- įjungtas Secret scanning ir Push protection, jei planas tai leidžia;
- Dependabot alerts ir security updates;
- Code scanning (CodeQL arba tinkamas analizatorius);
- SECURITY.md su pranešimo kanalu;
- CODEOWNERS jautriems katalogams;
- branch protection arba ruleset main šakai;
- privalomi testai prieš merge;
- bent viena nepriklausoma PR peržiūra jautriems pakeitimams;
- draudimas force-push ir branch trynimo main šakoje;
- pasirašytų commitų arba žymų reikalavimas ten, kur tai įmanoma;
- aiškus release ir rollback procesas.

6. Branch, PR ir release tvarka

1. Sukurk trumpai gyvenančią feature arba fix šaką.
2. Pakeitimą padaryk mažą ir lengvai peržiūrimą.
3. Paleisk testus, lint, `git diff --check` ir saugumo skenavimą.
4. Atidaryk PR su problema, sprendimu, rizika ir validacija.
5. Po review merge į main tik per apsaugotą taisyklę.
6. Sukurk versijos tagą, hash manifestą ir, jei reikia, pasirašyk release.
7. Viešai skelbk tik tai, kas patikrinta ir neturi paslapčių.

7. Incidento planas nutekėjus raktui

- **PAT/API tokenas:** revoke dabar; nebandyk jo „tiesiog paslėpti“.
- **SSH privatus raktas:** pašalink viešą raktą iš GitHub, sugeneruok naują porą, atnaujink deploy vietas.
- **Release signing key:** pažymėk seną raktą revoked, paskelbk naują viešą raktą ir naują fingerprintą.
- **Cloud, DNS, Telegram ar kitos paslaptys:** revoke/rotate pas tiekėją.
- **Git istorija:** po revokacijos pašalink aptiktą paslaptį iš istorijos ir patikrink visas šakas, tagus, forkus bei Actions logus.
- **Auditas:** patikrink GitHub audit log, prisijungimus, workflow paleidimus ir netikėtus release.

8. SafeStack rekomenduojama kredencijų matrica

Paskirtis	Rekomenduojamas mechanizmas	Prieiga
Kasdienis git push	SSH user key	tik konkreči paskyra
Vieno repo deployment	atskiras deploy key	vienas repo, dažniausiai read-only
CI tame pačiame repo	GITHUB_TOKEN	minimalūs workflow leidimai

Paskirtis	Rekomenduojamas mechanizmas	Prieiga
Organizacijos automatizacija	GitHub App	konkretūs repo ir permission
Vienkartinis API testas	fine-grained PAT	konkretus repo, trumpas laikas
Release autentiškumas	atskiras signing key	privatus raktas offline
DNS / Telegram / cloud	tiekėjo secret	tik atitinkama paslauga

9. Kas niekada neturi būti vieša

- privatūs SSH, GPG, S/MIME ar release signing raktai;
- PAT, OAuth, GitHub App private key ir JWT paslaptys;
- Cloudflare, DNS, Telegram, SMTP, database ir cloud kredencialai;
- slaptažodžiai, .env failai ir atsarginės kopijos su paslaptimis;
- production konfigūracijos, jei jose yra autentikacija;
- debug logai su Authorization, cookies ar naudotojų duomenimis.

10. Oficialūs šaltiniai

- [GitHub autentikacija](#)
- [PAT valdymas](#)
- [Fine-grained PAT leidimai](#)
- [GitHub credential tipų lentelė](#)
- [GitHub Apps ir OAuth skirtumai](#)
- [REST API rate limits](#)
- [Actions secrets](#)
- [Secret scanning](#)
- [Push protection](#)
- [Commit pasirašymas](#)
- [GitHub SSH fingerprintai](#)

Atnaujinimas: 2026-09-20